Rapport de Projet

Système de Gestion de Pharmacie

Application Java avec Interface Graphique JavaFX

Développement d'une solution complète pour la gestion quotidienne d'une pharmacie

Projet réalisé par :

Tasnim Abid
Wala Ouaili

Classe : IOT2

23 janvier 2026

Table des matières

1 Introduction et Contexte du Projet **3**
- 1.1 Présentation Générale . . . 3
  - 1.1.1 Problématique . . . 3
  - 1.1.2 Objectifs du Projet . . . 3
- 1.2 Technologies Choisies . . . 4
  - 1.2.1 Java comme Langage Principal . . . 4
  - 1.2.2 JavaFX pour l'Interface Graphique . . . 4
  - 1.2.3 Oracle Database . . . 4

2 Diagramme de Cas d'Utilisation **5**
- 2.1 Vue d'ensemble . . . 5
- 2.2 Fonctionnalités principales . . . 5
  - 2.2.1 Fonctionnalités du Pharmacien . . . 5
  - 2.2.2 Fonctionnalités du Gestionnaire . . . 6

3 Description des Classes Principales **7**
- 3.1 Classe Utilisateur . . . 7
  - 3.1.1 Description générale . . . 7
- 3.2 Classe Pharmacien . . . 8
  - 3.2.1 Description générale . . . 8
  - 3.2.2 Méthodes principales . . . 8
- 3.3 Classe Gestionnaire . . . 13
  - 3.3.1 Description générale . . . 13
  - 3.3.2 Méthodes principales . . . 13
- 3.4 Classe Medicament . . . 19
  - 3.4.1 Description . . . 19
- 3.5 Classe Vente . . . 20
  - 3.5.1 Description . . . 20
- 3.6 Classe StockHistorique . . . 21
  - 3.6.1 Description . . . 21
- 3.7 Classe Commande . . . 23
  - 3.7.1 Description . . . 23
- 3.8 Classe DAO . . . 24
  - 3.8.1 Description . . . 24
- 3.9 Classe DatabaseConnection . . . 25
  - 3.9.1 Description . . . 25
- 3.10 Classe Main . . . 26

3.10.1 Description 26

4 Interface Graphique JavaFX 29
4.1 Vue d'ensemble 29
4.2 Principales caractéristiques 29

5 Base de Données de Gestion de Pharmacie 30
5.1 Structure de la base de données 30
5.1.1 Table Pharmacien 30
5.1.2 Table Vente 30
5.1.3 Table Commande 31
5.1.4 Table StockHistorique 31
5.2 Données de test 32

6 Améliorations Proposées 33
6.1 Court Terme 33
6.1.1 Sécurité 33
6.1.2 UI/UX 33
6.1.3 Robustesse 33
6.2 Moyen Terme 33
6.2.1 Fonctionnalités avancées 33
6.2.2 Architecture 33
6.2.3 Données 34
6.3 Long Terme 34
6.3.1 Intelligence artificielle 34
6.3.2 Intégration 34
6.3.3 Cloud 34

7 Conclusion Finale 35

Chapitre 1

Introduction et Contexte du Projet

1.1 Présentation Générale

Le système de gestion de pharmacie représente une solution informatique complète destinée à automatiser et optimiser les opérations quotidiennes d'une officine pharmaceutique. Dans un contexte où la rigueur, la traçabilité et l'efficacité sont primordiales, ce projet vise à fournir un outil moderne et fiable pour gérer l'ensemble des processus métier d'une pharmacie.

1.1.1 Problématique

Les pharmacies modernes font face à plusieurs défis opérationnels :

— **Gestion du stock :** Nécessité de maintenir un inventaire précis et à jour pour éviter les ruptures de stock critiques

— **Traçabilité des ventes :** Obligation légale de conserver l'historique complet des transactions

— **Réapprovisionnement :** Besoin d'anticiper les commandes pour assurer la continuité du service

— **Gestion multi-utilisateurs :** Coordination entre pharmaciens et gestionnaires avec des droits différenciés

— **Reporting :** Production de rapports statistiques pour l'analyse d'activité et la prise de décision

1.1.2 Objectifs du Projet

L'application développée répond à ces besoins en proposant :

1. Un système d'authentification sécurisé avec gestion des rôles
2. Une interface intuitive pour l'enregistrement rapide des ventes
3. Un suivi en temps réel des stocks avec alertes automatiques
4. Un module de commandes pour le réapprovisionnement
5. Des outils de reporting et d'analyse statistique
6. Un historique complet des mouvements de stock

1.2 Technologies Choisies

1.2.1 Java comme Langage Principal

Le choix de Java s'est imposé pour plusieurs raisons :

Portabilité : Application exécutable sur différents systèmes d'exploitation

Orienté Objet : Facilite la modélisation des entités métier (Pharmacien, Médicament, Vente...)

Robustesse : Gestion forte du typage et des exceptions

Écosystème riche : Bibliothèques disponibles pour JDBC, JavaFX, etc.

Performance : JVM optimisée pour les applications d'entreprise

1.2.2 JavaFX pour l'Interface Graphique

JavaFX a été retenu pour l'interface utilisateur en raison de :

— Son intégration native avec Java

— Ses capacités de création d'interfaces modernes et esthétiques

— Son système de gestion d'événements intuitif

— Sa flexibilité pour le design responsive

1.2.3 Oracle Database

La base de données Oracle a été sélectionnée pour :

— Sa fiabilité et sa stabilité reconnues en entreprise

— Son support complet du SQL standard

— Ses outils de gestion et d'administration

— Sa capacité à gérer de grands volumes de données

Chapitre 2

Diagramme de Cas d'Utilisation

2.1 Vue d'ensemble

Ce diagramme de cas d'utilisation représente les interactions entre les deux types d'utilisateurs du système de gestion de pharmacie : le pharmacien et le gestionnaire. Chaque acteur est associé à un ensemble de fonctionnalités spécifiques, comme l'enregistrement ou l'annulation des ventes pour le pharmacien, et la gestion des commandes ou la génération de rapports pour le gestionnaire.

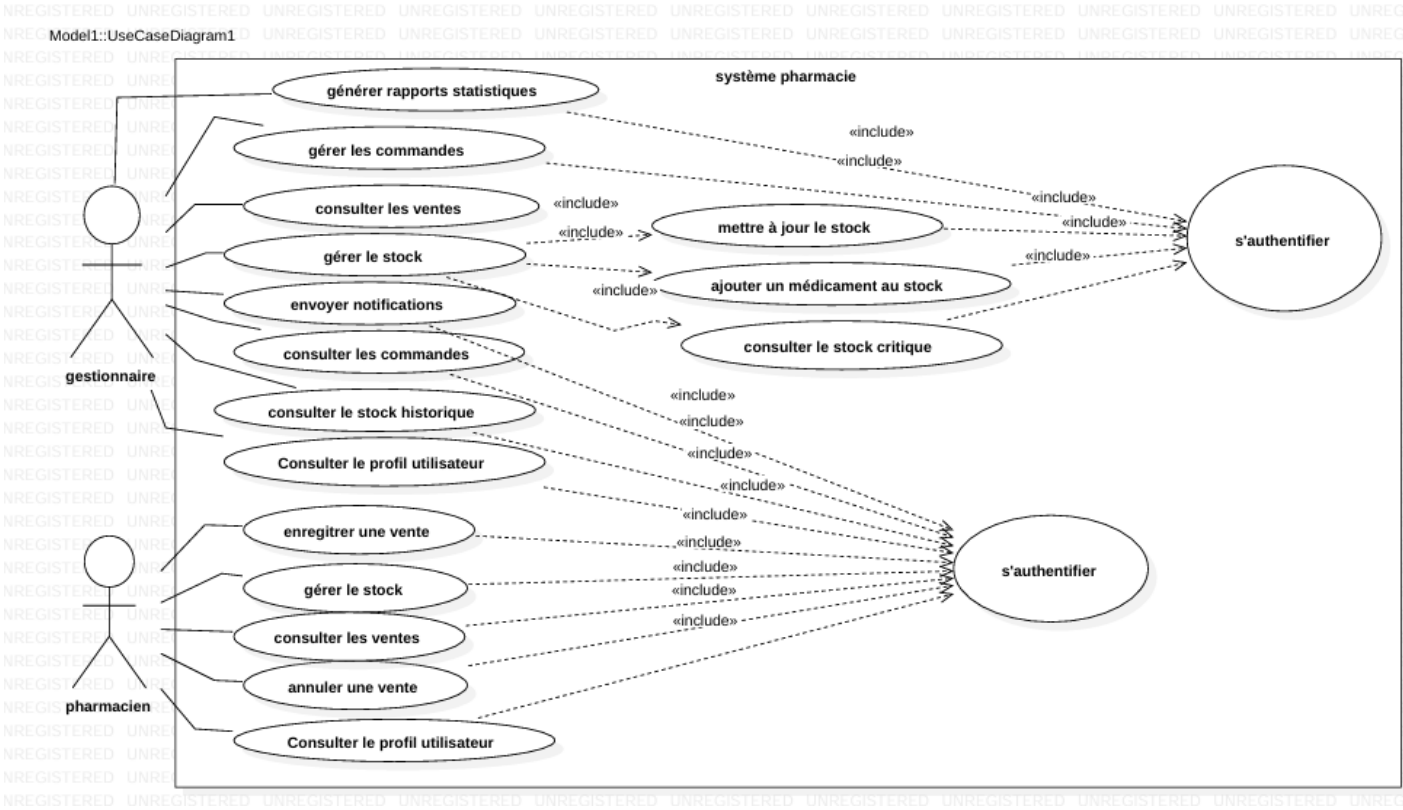


FIGURE 2.1 – Diagramme de cas d'utilisation du système de gestion de pharmacie

2.2 Fonctionnalités principales

2.2.1 Fonctionnalités du Pharmacien

— Enregistrer une vente

— Annuler une vente
— Consulter les ventes du jour
— Consulter le stock
— Consulter le profil utilisateur

2.2.2 Fonctionnalités du Gestionnaire

— Gérer le stock (ajouter, mettre à jour, consulter stock critique)
— Passer des commandes
— Générer des rapports
— Consulter l'historique des mouvements
— Consulter le profil utilisateur

Certaines fonctionnalités sont partagées, comme la consultation du stock ou du profil utilisateur. Plusieurs cas d'utilisation incluent des sous-fonctionnalités, par exemple « gérer le stock » inclut « mettre à jour le stock », « ajouter un médicament » et « consulter le stock critique ». Le diagramme montre aussi que l'authentification est une étape préalable incluse dans toutes les actions.

Chapitre 3

Description des Classes Principales

3.1 Classe Utilisateur

3.1.1 Description générale

La classe `Utilisateur` constitue la base abstraite de tous les types d'utilisateurs du système de gestion de pharmacie. Elle définit les informations essentielles d'un utilisateur, telles que son identifiant, son nom, son prénom, son login et son mot de passe, tout en garantissant l'encapsulation grâce aux getters et setters.

En plus de servir de modèle commun, elle impose aux classes dérivées (comme `Pharmacien`, `Gestionnaire`) d'implémenter une méthode d'authentification, ce qui assure que chaque type d'utilisateur possède sa propre logique de connexion à la base de données.

```java
package pharm;
import java.sql.Connection;
import java.sql.SQLException;

public abstract class Utilisateur {
    protected int id;
    protected String nom;
    protected String prenom;
    protected String login;
    protected String pwd;

    public Utilisateur(int id, String nom, String prenom, String login, String pwd) {
        this.id = id;
        this.nom = nom;
        this.prenom = prenom;
        this.login = login;
        this.pwd = pwd;
    }

    // Getters and Setters
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public String getPrenom() {
        return prenom;
    }
```

FIGURE 3.1 – code utilisateur

3.2 Classe Pharmacien

3.2.1 Description générale

La classe `Pharmacien` hérite de la classe abstraite `Utilisateur` et représente l'acteur principal du système, chargé de gérer les opérations quotidiennes de la pharmacie. Elle implémente la méthode d'authentification pour vérifier l'identité du pharmacien dans la base de données.

3.2.2 Méthodes principales

Constructeur

Le constructeur de la classe `Pharmacien` permet d'initialiser un nouvel objet pharmacien avec ses informations de base : identifiant, nom, prénom, login et mot de passe. Il utilise le mot-clé `super` pour appeler le constructeur de la classe parente `Utilisateur`, ce qui garantit que toutes les données héritées sont correctement enregistrées dès la création

de l'objet.

Méthode authentifier()

La méthode `authentifier` permet de vérifier l'identité du pharmacien en comparant le login et le mot de passe saisis avec ceux enregistrés dans la base de données. Elle exécute une requête SQL qui recherche un pharmacien correspondant dans la table `Pharmacien`.

Si une correspondance est trouvée, elle récupère son identifiant, son nom et son prénom, puis les affecte aux attributs de l'objet courant, ce qui confirme que l'utilisateur est bien reconnu par le système. Dans ce cas, la méthode retourne `true`, indiquant une authentification réussie ; sinon, elle retourne `false`.

```java
package pharm;
import java.sql.Connection;
import java.sql.SQLException;

public abstract class Utilisateur {
    protected int id;
    protected String nom;
    protected String prenom;
    protected String login;
    protected String pwd;

    public Utilisateur(int id, String nom, String prenom, String login, String pwd) {
        this.id = id;
        this.nom = nom;
        this.prenom = prenom;
        this.login = login;
        this.pwd = pwd;
    }

    // Getters and Setters
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public String getPrenom() {
        return prenom;
    }
```

FIGURE 3.2 – code authentifier

Méthode enregistrerVente()

La méthode `enregistrerVente` permet au pharmacien de réaliser une transaction complète en assurant la cohérence entre les ventes et le stock. Elle commence par demander les informations nécessaires à la vente, telles que l'identifiant du médicament, l'identifiant du client (ou zéro si inconnu) et la quantité souhaitée.

Ensuite, elle vérifie dans la base de données que le médicament existe et que le stock est suffisant pour couvrir la demande. Si les conditions sont respectées, la vente est enregistrée dans la table `Vente` avec toutes les informations pertinentes, y compris la date et l'identité du pharmacien.

Pour générer automatiquement un identifiant unique de vente, la requête utilise `seq_vente.NEXTVAL`, qui correspond à une séquence Oracle permettant d'incrémenter et d'attribuer un nouvel identifiant à chaque enregistrement sans risque de duplication.

Le stock du médicament est alors décrémenté de la quantité vendue, et une trace est ajoutée dans la table `StockHistorique` afin de conserver un suivi précis des mouvements. Enfin, la transaction est validée par un `commit`, garantissant que toutes les opérations sont confirmées, ou annulée par un `rollback` en cas d'erreur.

```java
public void enregistrerVente(Connection conn, Scanner scanner) {
    try {
        System.out.print("ID du médicament : ");
        int idMed = Integer.parseInt(scanner.nextLine());

        System.out.print("ID du client (ou 0 si inconnu) : ");
        int idClient = Integer.parseInt(scanner.nextLine());

        System.out.print("Quantité : ");
        int quantite = Integer.parseInt(scanner.nextLine());

        // Vérifier stock suffisant
        String checkSql = "SELECT stock FROM Medicament WHERE id_medicament = ?";
        try (PreparedStatement check = conn.prepareStatement(checkSql)) {
            check.setInt(1, idMed);
            try (ResultSet rs = check.executeQuery()) {
                if (!rs.next() || rs.getInt("stock") < quantite) {
                    System.out.println("Stock insuffisant ou médicament inexistant !");
                    return;
                }
            }
        }

        // Insérer la vente
        String insertVente = "INSERT INTO Vente " +
                "(id_vente, id_pharmacien, id_client, id_medicament, quantite, date_vente) " +
                "VALUES (seq_vente.NEXTVAL, ?, ?, ?, ?, SYSDATE)";

        try (PreparedStatement ps = conn.prepareStatement(insertVente)) {
            ps.setInt(1, this.id);
            if (idClient > 0) {
                ps.setInt(2, idClient);
            } else {
                ps.setNull(2, java.sql.Types.NUMERIC);
            }
            ps.setInt(3, idMed);
            ps.setInt(4, quantite);
            ps.executeUpdate();
        }

        // Mettre à jour stock
        String updateStock = "UPDATE Medicament SET stock = stock - ? WHERE id_medicament = ?";
        try (PreparedStatement ps = conn.prepareStatement(updateStock)) {
            ps.setInt(1, quantite);
            ps.setInt(2, idMed);
            ps.executeUpdate();
        }

        // Historique
        String hist ="INSERT INTO StockHistorique " +
                "(id_historique, id_medicament, quantite, date_mouvement, type_mouvement) " +
                "VALUES (seq_historique.NEXTVAL, ?, ?, SYSDATE, 'VENTE')";

        try (PreparedStatement ps = conn.prepareStatement(hist)) {
            ps.setInt(1, idMed);
            ps.setInt(2, -quantite);
            ps.executeUpdate();
        }

        conn.commit();
        System.out.println("Vente enregistrée avec succès !");

    } catch (Exception e) {
        System.out.println("Erreur lors de la vente : " + e.getMessage());
        try { conn.rollback(); } catch (Exception ignored) {}
```

FIGURE 3.3 – code enregistervente()

Méthode consulterVentesDuJour()

La méthode `consulterVentesDuJour` permet au pharmacien de consulter toutes les ventes qu'il a réalisées au cours de la journée. Elle exécute une requête SQL qui sélectionne, dans la table `Vente`, les ventes associées à l'identifiant du pharmacien connecté et dont la date correspond à la date du jour (`TRUNC(SYSDATE)`).

La requête joint également la table `Medicament` pour afficher le nom du produit vendu et la table `Client` pour identifier le client, si celui-ci est enregistré. Les résultats sont parcourus et affichés sous forme de liste détaillée, indiquant l'ID de la vente, le nom du médicament, la quantité vendue, le nom du client (ou "Inconnu" si non renseigné) ainsi que la date de la transaction.

```
public void consulterVentesDuJour(Connection conn) {
    String sql = "SELECT v.id_vente, m.nom, v.quantite, v.date_vente, c.nom || ' ' || c.prenom as client " +
                 "FROM Vente v JOIN Medicament m ON v.id_medicament = m.id_medicament " +
                 "LEFT JOIN Client c ON v.id_client = c.id_client " +
                 "WHERE v.id_pharmacien = ? AND TRUNC(v.date_vente) = TRUNC(SYSDATE)";

    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, this.id);
        try (ResultSet rs = ps.executeQuery()) {
            System.out.println("\nVentes du jour :");
            while (rs.next()) {
                System.out.printf("ID: %d | Médicament: %-15s | Qté: %d | Client: %s | Date: %s\n",
                        rs.getInt("id_vente"), rs.getString("nom"), rs.getInt("quantite"),
                        rs.getString("client") != null ? rs.getString("client") : "Inconnu",
                        rs.getTimestamp("date_vente"));
            }
        }
    } catch (SQLException e) {
        System.out.println("Erreur consultation ventes : " + e.getMessage());
    }
}
```

FIGURE 3.4 – code consulterventedujour()

Méthode annulerVente()

La méthode `annulerVente` permet au pharmacien d'annuler une transaction déjà enregistrée et de rétablir l'état du stock. Elle commence par demander l'identifiant de la vente à annuler, puis vérifie dans la table `Vente` que cette vente existe et qu'elle appartient bien au pharmacien connecté.

Si la vente est trouvée, elle récupère les informations essentielles, notamment l'identifiant du médicament et la quantité vendue. Ensuite, la vente est supprimée de la table `Vente`, et le stock du médicament concerné est incrémenté de la quantité annulée afin de restituer les produits.

Pour assurer une traçabilité complète, un enregistrement est ajouté dans la table `StockHistorique` avec le type de mouvement `ANNULATION_VENTE`, ce qui permet de garder une trace claire de l'opération.

```java
public void annulerVente(Connection conn, Scanner scanner) {
    System.out.print("ID de la vente à annuler : ");
    try {
        int idVente = Integer.parseInt(scanner.nextLine());

        // Récupérer infos vente
        String getVente = "SELECT id_medicament, quantite FROM Vente WHERE id_vente = ? AND id_pharmacien = ?";
        int idMed = 0, qte = 0;
        try (PreparedStatement ps = conn.prepareStatement(getVente)) {
            ps.setInt(1, idVente);
            ps.setInt(2, this.id);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) {
                    System.out.println("Vente non trouvée ou non autorisée.");
                    return;
                }
                idMed = rs.getInt("id_medicament");
                qte = rs.getInt("quantite");
            }
        }

        // Supprimer vente
        String delete = "DELETE FROM Vente WHERE id_vente = ?";
        try (PreparedStatement ps = conn.prepareStatement(delete)) {
            ps.setInt(1, idVente);
            ps.executeUpdate();
        }

        // Remettre stock
        String update = "UPDATE Medicament SET stock = stock + ? WHERE id_medicament = ?";
        try (PreparedStatement ps = conn.prepareStatement(update)) {
            ps.setInt(1, qte);
            ps.setInt(2, idMed);
            ps.executeUpdate();
        }

        // Historique
        String hist = "INSERT INTO StockHistorique " +
                "(id_historique, id_medicament, quantite, date_mouvement, type_mouvement) " +
                "VALUES (seq_historique.NEXTVAL, ?, ?, SYSDATE, 'ANNULATION_VENTE')";

        try (PreparedStatement ps = conn.prepareStatement(hist)) {
            ps.setInt(1, idMed);
            ps.setInt(2, qte);
            ps.executeUpdate();
```

FIGURE 3.5 – code annulervente()

3.3 Classe Gestionnaire

3.3.1 Description générale

La classe `Gestionnaire` hérite également de la classe `Utilisateur` et représente l'acteur responsable de la supervision et de l'organisation du stock.

3.3.2 Méthodes principales

Constructeur

Il sert à créer un nouvel objet gestionnaire en initialisant ses informations de base (id, nom, prénom, login et mot de passe). Grâce au mot-clé `super`, il appelle le constructeur de la classe `Utilisateur` pour enregistrer ces données héritées dès la création de l'objet.

Méthode authentifier()

Elle permet de vérifier l'identité du gestionnaire en comparant le login et le mot de passe saisis avec ceux enregistrés dans la table `Gestionnaire`. Si une correspondance est trouvée, l'objet est mis à jour avec l'id, le nom et le prénom du gestionnaire, et la méthode retourne `true`. Sinon, elle retourne `false`.

```
1  package pharm;
2
3  import java.sql.Connection;
8
9  public class Gestionnaire extends Utilisateur {
10
11     public Gestionnaire(int id, String nom, String prenom, String login, String pwd) {
12         super(id, nom, prenom, login, pwd);
13     }
14
15     @Override
16     public boolean authentifier(String loginSaisi, String pwdSaisi, Connection conn) throws SQLException {
17         String sql = "SELECT id_gestionnaire, nom, prenom FROM Gestionnaire WHERE login = ? AND pwd = ?";
18         try (PreparedStatement ps = conn.prepareStatement(sql)) {
19             ps.setString(1, loginSaisi);
20             ps.setString(2, pwdSaisi);
21             try (ResultSet rs = ps.executeQuery()) {
22                 if (rs.next()) {
23                     this.id = rs.getInt("id_gestionnaire");
24                     this.nom = rs.getString("nom");
25                     this.prenom = rs.getString("prenom");
26                     return true;
27                 }
28                 return false;
29             }
30         }
31     }
```

FIGURE 3.6 – code authentifier()

Méthode ajouterMedicament()

La méthode `ajouterMedicament` permet au gestionnaire d'ajouter un nouveau médicament dans la base en saisissant son nom, son dosage, son stock initial et son prix unitaire. L'insertion utilise `seq_medicament.NEXTVAL` afin de générer automatiquement un identifiant unique pour chaque produit. Une fois la requête exécutée, la transaction est validée par un `commit`, et un message confirme l'ajout.

```
    public void ajouterMedicament(Connection conn, Scanner scanner) {
        try {
            System.out.print("Nom du médicament : ");
            String nom = scanner.nextLine();

            System.out.print("Dosage : ");
            String dosage = scanner.nextLine();

            System.out.print("Stock initial : ");
            int stock = Integer.parseInt(scanner.nextLine());

            System.out.print("Prix unitaire : ");
            double prix = Double.parseDouble(scanner.nextLine());

            String sql ="INSERT INTO Medicament"+
                    "(id_medicament, nom, dosage, stock, prix_unitaire)"+
                    "VALUES (seq_medicament.NEXTVAL, ?, ?, ?, ?)";

            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setString(1, nom);
                ps.setString(2, dosage);
                ps.setInt(3, stock);
                ps.setDouble(4, prix);
                ps.executeUpdate();
            }

            conn.commit();
            System.out.println("Médicament ajouté.");

        } catch (Exception e) {
            System.out.println("Erreur ajout médicament : " + e.getMessage());
        }
    }
```

FIGURE 3.7 – code ajoutermedicament()

Méthode mettreAJourStock()

La méthode `mettreAJourStock` permet au gestionnaire de modifier manuellement la quantité disponible d'un médicament dans la base. Elle commence par demander l'identifiant du médicament et la nouvelle quantité de stock à enregistrer. Une requête SQL met ensuite à jour la table `Medicament` avec cette valeur.

Après la mise à jour, un enregistrement est ajouté dans la table `StockHistorique` avec le type de mouvement `AJUSTEMENT_MANUEL`, ce qui assure une traçabilité des changements effectués.

```java
public void mettreAJourStock(Connection conn, Scanner scanner) {
    try {
        System.out.print("ID du médicament : ");
        int id = Integer.parseInt(scanner.nextLine());

        System.out.print("Nouvelle quantité en stock : ");
        int nouveauStock = Integer.parseInt(scanner.nextLine());

        String sql = "UPDATE Medicament SET stock = ? WHERE id_medicament = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, nouveauStock);
            ps.setInt(2, id);
            int rows = ps.executeUpdate();
            if (rows == 0) {
                System.out.println("Médicament non trouvé.");
                return;
            }
        }

        // Historique
        String hist = "INSERT INTO StockHistorique (id_medicament, quantite, date_mouvement, type_mouvement) " +
                      "VALUES (?, ?, SYSDATE, 'AJUSTEMENT_MANUEL')";
        try (PreparedStatement ps = conn.prepareStatement(hist)) {
            ps.setInt(1, id);
            ps.setInt(2, nouveauStock); // ou delta si tu calcules
            ps.executeUpdate();
        }

        conn.commit();
        System.out.println("Stock mis à jour.");

    } catch (Exception e) {
        System.out.println("Erreur mise à jour stock : " + e.getMessage());
    }
}
```

FIGURE 3.8 – code mettreAJourStock()

Méthode consulterStockCritique()

La méthode `consulterStockCritique` permet au gestionnaire d'identifier rapidement les médicaments dont le stock est faible et risque d'entraîner une rupture. Elle exécute une requête SQL qui sélectionne tous les produits dont la quantité en stock est inférieure ou égale à 10, puis les affiche avec leurs principales informations : nom, dosage, stock restant et prix unitaire.

```java
public void consulterStockCritique(Connection conn) {
    String sql = "SELECT nom, dosage, stock, prix_unitaire FROM Medicament WHERE stock <= 10 ORDER BY stock";

    try (PreparedStatement ps = conn.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {

        System.out.println("\nMédicaments en stock critique (≤ 10 unités) :");
        boolean trouve = false;
        while (rs.next()) {
            trouve = true;
            System.out.printf("%-20s %-10s Stock: %d   Prix: %.2f TND\n",
                    rs.getString("nom"), rs.getString("dosage"),
                    rs.getInt("stock"), rs.getDouble("prix_unitaire"));

            // Simulation notification
            System.out.println("   → Notification envoyée : RUPTURE PROCHAINE - " + rs.getString("nom"));
        }
        if (!trouve) {
            System.out.println("Aucun médicament en stock critique.");
        }

    } catch (SQLException e) {
        System.out.println("Erreur consultation critique : " + e.getMessage());
    }
}
```

FIGURE 3.9 – code consulterStockCritique()

Méthode passerCommande()

La méthode `passerCommande` permet au gestionnaire d'enregistrer une commande de réapprovisionnement pour un médicament. Elle commence par demander l'identifiant du médicament et la quantité à commander, puis insère ces informations dans la table `Commande` avec l'ID du gestionnaire et la date de la commande.

Ensuite, le stock du médicament est directement augmenté de la quantité commandée, et un enregistrement est ajouté dans la table `StockHistorique` avec le type de mouvement `COMMANDE`, ce qui assure la traçabilité de l'opération.

```java
public void passerCommande(Connection conn, Scanner scanner) {
    try {
        System.out.print("ID du médicament à commander : ");
        int idMed = Integer.parseInt(scanner.nextLine());

        System.out.print("Quantité à commander : ");
        int qte = Integer.parseInt(scanner.nextLine());

        String sql = "INSERT INTO Commande " +
                "(id_commande, id_gestionnaire, id_medicament, quantite, date_commande) " +
                "VALUES (seq_commande.NEXTVAL, ?, ?, ?, SYSDATE)";

        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, this.id);
            ps.setInt(2, idMed);
            ps.setInt(3, qte);
            ps.executeUpdate();
        }

        // Mise à jour stock (simplification : on ajoute directement)
        String update = "UPDATE Medicament SET stock = stock + ? WHERE id_medicament = ?";
        try (PreparedStatement ps = conn.prepareStatement(update)) {
            ps.setInt(1, qte);
            ps.setInt(2, idMed);
            ps.executeUpdate();
        }

        // Historique
        String hist = "INSERT INTO StockHistorique " +
                "(id_historique, id_medicament, quantite, date_mouvement, type_mouvement) " +
                "VALUES (seq_historique.NEXTVAL, ?, ?, SYSDATE, 'COMMANDE')";

        try (PreparedStatement ps = conn.prepareStatement(hist)) {
            ps.setInt(1, idMed);
            ps.setInt(2, qte);
            ps.executeUpdate();
        }

        conn.commit();
        System.out.println("Commande enregistrée et stock mis à jour.");

    } catch (Exception e) {
        System.out.println("Erreur commande : " + e.getMessage());
    }
```

FIGURE 3.10 – code passerCommande()

Méthode consulterHistoriqueStock()

La méthode `consulterHistoriqueStock` permet au gestionnaire d'afficher tous les mouvements de stock enregistrés (ventes, commandes, ajustements, annulations). Elle récupère les données depuis la table `StockHistorique` en les liant aux médicaments concernés, puis les présente avec la quantité, le type de mouvement et la date.

```
    public void consulterHistoriqueStock(Connection conn) {
        String sql = "SELECT m.nom, h.quantite, h.date_mouvement, h.type_mouvement " +
                     "FROM StockHistorique h JOIN Medicament m ON h.id_medicament = m.id_medicament " +
                     "ORDER BY h.date_mouvement DESC";

        try (PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            System.out.println("\nHistorique des mouvements de stock :");
            while (rs.next()) {
                System.out.printf("%-20s | Qté: %+5d | Type: %-12s | %s\n",
                        rs.getString("nom"), rs.getInt("quantite"),
                        rs.getString("type_mouvement"), rs.getTimestamp("date_mouvement"));
            }

        } catch (SQLException e) {
            System.out.println("Erreur historique : " + e.getMessage());
        }
    }
}
```

FIGURE 3.11 – code : consulterhistoriquestock()

3.4 Classe Medicament

3.4.1 Description

La classe `Medicament` représente un médicament dans le système de gestion de pharmacie. Elle contient les principales informations nécessaires : identifiant, nom, dosage, quantité en stock et prix unitaire. Son constructeur permet d'initialiser ces données lors de la création d'un objet, et des méthodes getters offrent un accès sécurisé aux attributs.

Cette classe joue donc le rôle de modèle de données, servant à manipuler et afficher les informations relatives aux médicaments de manière simple et organisée.

```java
package pharm;

public class Medicament {
    private int id;
    private String nom;
    private String dosage;
    private int stock;
    private double prixUnitaire;

    public Medicament(int id, String nom, String dosage, int stock, double prixUnitaire) {
        this.id = id;
        this.nom = nom;
        this.dosage = dosage;
        this.stock = stock;
        this.prixUnitaire = prixUnitaire;
    }

    // Getters
    public int getId() { return id; }
    public String getNom() { return nom; }
    public String getDosage() { return dosage; }
    public int getStock() { return stock; }
    public double getPrixUnitaire() { return prixUnitaire; }
}

```

FIGURE 3.12 – code classe medicament

3.5 Classe Vente

3.5.1 Description

La classe `Vente` représente une transaction réalisée dans la pharmacie. Elle regroupe toutes les informations nécessaires à une vente : l'identifiant de la vente, le pharmacien qui l'a effectuée, le client concerné, le médicament vendu, la quantité ainsi que la date de la transaction.

Son constructeur permet d'initialiser ces données lors de la création d'un objet, tandis que les getters et setters offrent un accès et une modification sécurisée des attributs. Cette classe joue donc le rôle de modèle de données pour les ventes, facilitant la gestion, l'affichage et la traçabilité des opérations effectuées par les pharmaciens.

```java
package pharm;

//Vente.java
import java.util.Date;

public class Vente {
private int id;
private int idPharmacien;
private int idClient;
private int idMedicament;
private int quantite;
private Date date;

public Vente(int id, int idPharmacien, int idClient, int idMedicament, int quantite, Date date) {
    this.id = id;
    this.idPharmacien = idPharmacien;
    this.idClient = idClient;
    this.idMedicament = idMedicament;
    this.quantite = quantite;
    this.date = date;
}

// Getters and Setters
public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public int getIdPharmacien() {
    return idPharmacien;
}

public void setIdPharmacien(int idPharmacien) {
    this.idPharmacien = idPharmacien;
}

public int getIdClient() {
    return idClient;
}

public void setIdClient(int idClient) {
    this.idClient = idClient;
```

FIGURE 3.13 – code classe vente

3.6 Classe StockHistorique

3.6.1 Description

La classe `StockHistorique` sert à enregistrer et suivre les mouvements de stock liés aux médicaments. Elle contient l'identifiant du médicament concerné, la quantité ajoutée ou retirée, ainsi que la date du mouvement. Son constructeur initialise ces données, et les getters/setters permettent d'y accéder ou de les modifier.

Cette classe joue donc un rôle de traçabilité, en gardant une mémoire claire des ajustements et opérations effectués sur le stock.

```java
package pharm;

//StockHistorique.java
import java.util.Date;

public class StockHistorique {
 private int idMedicament;
 private int quantite;
 private Date date;

 public StockHistorique(int idMedicament, int quantite, Date date) {
     this.idMedicament = idMedicament;
     this.quantite = quantite;
     this.date = date;
 }

 // Getters and Setters
 public int getIdMedicament() {
     return idMedicament;
 }

 public void setIdMedicament(int idMedicament) {
     this.idMedicament = idMedicament;
 }

 public int getQuantite() {
     return quantite;
 }

 public void setQuantite(int quantite) {
     this.quantite = quantite;
 }

 public Date getDate() {
     return date;
 }

 public void setDate(Date date) {
     this.date = date;
 }
}
```

FIGURE 3.14 – code classe stockhistorique

3.7 Classe Commande

3.7.1 Description

La classe `Commande` représente une commande de réapprovisionnement effectuée par un gestionnaire. Elle contient l'identifiant de la commande, du gestionnaire, du médicament concerné, la quantité commandée ainsi que la date. Son constructeur initialise ces données, et les getters/setters permettent d'y accéder ou de les modifier.

Cette classe sert donc de modèle pour gérer et tracer les commandes dans le système.

```java
package pharm;

//Commande.java
import java.util.Date;

public class Commande {
private int id;
private int idGestionnaire;
private int idMedicament;
private int quantite;
private Date date;

public Commande(int id, int idGestionnaire, int idMedicament, int quantite, Date date) {
    this.id = id;
    this.idGestionnaire = idGestionnaire;
    this.idMedicament = idMedicament;
    this.quantite = quantite;
    this.date = date;
}

// Getters and Setters
public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public int getIdGestionnaire() {
    return idGestionnaire;
}

public void setIdGestionnaire(int idGestionnaire) {
    this.idGestionnaire = idGestionnaire;
}

public int getIdMedicament() {
    return idMedicament;
}

public void setIdMedicament(int idMedicament) {
    this.idMedicament = idMedicament;
}

```

FIGURE 3.15 – code classe commande

3.8 Classe DAO

3.8.1 Description

La classe `DAO` (Data Access Object) joue le rôle d'intermédiaire entre l'application et la base de données. Elle regroupe les méthodes nécessaires pour effectuer des opérations courantes :

— `getAllMedicaments` : récupère la liste complète des médicaments depuis la base et les retourne sous forme de collection (Vector).

— `addMedicament` : insère un nouveau médicament dans la table `Medicament` avec ses informations (id, nom, dosage, stock, prix unitaire).

— `getVentesByMedicament` : retourne toutes les ventes associées à un médicament donné, en construisant des objets `Vente` à partir des données de la table `Vente`.

En résumé, cette classe centralise l'accès aux données et facilite la manipulation des objets métiers (`Medicament`, `Vente`) sans que le reste du programme ait à gérer directement les requêtes SQL.

```java
package pharm;

import java.sql.Connection;

public class DAO {

    public Vector<Medicament> getAllMedicaments(Connection conn) throws SQLException {
        Vector<Medicament> liste = new Vector<>();
        String sql = "SELECT id_medicament, nom, dosage, stock, prix_unitaire FROM Medicament ORDER BY nom";

        try (PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                Medicament m = new Medicament(
                        rs.getInt("id_medicament"),
                        rs.getString("nom"),
                        rs.getString("dosage"),
                        rs.getInt("stock"),
                        rs.getDouble("prix_unitaire")
                );
                liste.add(m);
            }
        }
        return liste;
    }
public void addMedicament(Medicament med, Connection conn) throws SQLException {
    String query = "INSERT INTO Medicament (id_medicament, nom, dosage, stock, prix_unitaire) VALUES (?, ?, ?, ?, ?)";
    PreparedStatement stmt = conn.prepareStatement(query);
    stmt.setInt(1, med.getId());
    stmt.setString(2, med.getNom());
    stmt.setString(3, med.getDosage());
    stmt.setInt(4, med.getStock());
    stmt.setDouble(5, med.getPrixUnitaire());
    stmt.executeUpdate();
}

// Get sales by medicament using Vector
public Vector<Vente> getVentesByMedicament(int idMedicament, Connection conn) throws SQLException {
    Vector<Vente> ventes = new Vector<>();
    String query = "SELECT * FROM Vente WHERE id_medicament = ?";
    PreparedStatement stmt = conn.prepareStatement(query);
    stmt.setInt(1, idMedicament);
    ResultSet rs = stmt.executeQuery();
    while (rs.next()) {
        Vente vente = new Vente(
            rs.getInt("id_vente"),
            rs.getInt("id_pharmacien"),
            rs.getInt("id_client"),
            rs.getInt("id_medicament"),
            rs.getInt("quantite"),
            rs.getDate("date")
        );
        ventes.add(vente);
    }
    return ventes;
}
}

```

FIGURE 3.16 – code classe DAO

3.9 Classe DatabaseConnection

3.9.1 Description

La classe `DatabaseConnection` gère la connexion entre l'application et la base de données Oracle. Elle définit l'URL, l'utilisateur et le mot de passe nécessaires, puis fournit une méthode `getConnection` qui établit la connexion et désactive l'auto-commit pour mieux contrôler les transactions.

Un petit test dans la méthode `main` permet de vérifier rapidement si la connexion est réussie ou d'afficher une erreur en cas de problème.

```java
package pharm;

import java.sql.Connection;

public class DatabaseConnection {

    private static final String URL = "jdbc:oracle:thin:@localhost:1521:xe";
    private static final String USER = "PHARM";
    private static final String PASSWORD = "IOT2";

    public static Connection getConnection() throws SQLException {


        Connection conn = DriverManager.getConnection(URL, USER, PASSWORD);
        conn.setAutoCommit(false);
        return conn;

    }

    // Test rapide
    public static void main(String[] args) {
        try (Connection conn = getConnection()) {
            System.out.println("Connexion Oracle réussie !");
        } catch (SQLException e) {
            System.err.println("Erreur connexion : " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

FIGURE 3.17 – code classe DB

3.10 Classe Main

3.10.1 Description

La classe `Main` est le point d'entrée du programme de gestion de pharmacie. Elle établit la connexion avec la base de données, puis lance une interface en console où l'utilisateur peut s'authentifier soit comme pharmacien, soit comme gestionnaire.

- Si l'utilisateur est **pharmacien**, il accède à un menu pour enregistrer des ventes, consulter les ventes du jour, vérifier le stock ou annuler une vente.
- Si l'utilisateur est **gestionnaire**, il dispose d'un menu pour ajouter des médicaments, mettre à jour le stock, consulter les stocks critiques, passer des commandes, consulter l'historique et générer un rapport des ventes.
- Des fonctions utilitaires comme `afficherStock` et `genererRapportVentes` facilitent l'affichage des informations.

```
========================================
   BIENVENUE DANS LA GESTION PHARMACIE
========================================

Login : karim
Mot de passe : pharm2025

Bienvenue Karim Ben Ali (Pharmacien)

--- Menu Pharmacien ---
1. Enregistrer une vente
2. Consulter les ventes du jour
3. Consulter le stock actuel
4. Annuler une vente (remboursement stock)
5. Déconnexion
Votre choix :
```

FIGURE 3.18 – execution menu pharmacien

```
Votre choix : 5
Déconnexion...

Login : gestionnaire1
Mot de passe : admin

Bienvenue Sophie Martin (Gestionnaire)

--- Menu Gestionnaire ---
1. Ajouter un médicament
2. Mettre à jour le stock d'un médicament
3. Consulter les stocks critiques
4. Passer une commande de réapprovisionnement
5. Consulter l'historique des stocks
6. Générer un rapport simple des ventes
7. Déconnexion
Votre choix :
```

FIGURE 3.19 – execution menu gestionnaire

Chapitre 4

Interface Graphique JavaFX

4.1 Vue d'ensemble

L'application dispose également d'une interface graphique moderne développée avec JavaFX, offrant une alternative visuelle à l'interface console. Cette interface permet une utilisation plus intuitive et agréable du système.

4.2 Principales caractéristiques

— Écran de connexion avec authentification sécurisée
— Menus spécifiques pour Pharmacien et Gestionnaire
— Dialogues modaux pour les opérations (vente, ajout médicament, etc.)
— Affichage des stocks et rapports dans des tableaux formatés
— Design moderne avec palette de couleurs professionnelle
— Feedback utilisateur via alertes de succès/erreur

Chapitre 5

Base de Données de Gestion de Pharmacie

5.1 Structure de la base de données

5.1.1 Table Pharmacien

Table qui stocke les informations des pharmaciens : identifiant, nom, prénom, login et mot de passe.

— La séquence `seq_medicament` gère les IDs

— Trois médicaments sont insérés (dont un avec stock bas pour tester les alertes)

```
CREATE TABLE Pharmacien (
    id_pharmacien   NUMBER          PRIMARY KEY,
    nom             VARCHAR2(50)    NOT NULL,
    prenom          VARCHAR2(50)    NOT NULL,
    login           VARCHAR2(50)    UNIQUE NOT NULL,
    pwd             VARCHAR2(100)   NOT NULL
);
```

FIGURE 5.1 – table pharmacien

5.1.2 Table Vente

Table qui enregistre les ventes réalisées par les pharmaciens.

— Champs : identifiant de la vente, pharmacien, client, médicament, quantité et date

— Contraintes : quantité $>$ 0 et relations avec les tables `Pharmacien`, `Client` et `Medicament`

— La séquence `seq_vente` génère les IDs

```
-- 5. VENTE
CREATE TABLE Vente (
    id_vente        NUMBER          PRIMARY KEY,
    id_pharmacien   NUMBER          NOT NULL,
    id_client       NUMBER,
    id_medicament   NUMBER          NOT NULL,
    quantite        NUMBER          NOT NULL    CHECK (quantite > 0),
    date_vente      TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_vente_pharmacien   FOREIGN KEY (id_pharmacien)   REFERENCES Pharmacien(id_pharmacien),
    CONSTRAINT fk_vente_client       FOREIGN KEY (id_client)       REFERENCES Client(id_client),
    CONSTRAINT fk_vente_medicament   FOREIGN KEY (id_medicament)   REFERENCES Medicament(id_medicament)
);

CREATE SEQUENCE seq_vente START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
```

Activer Win
Accédez aux pa

FIGURE 5.2 – table vente

5.1.3 Table Commande

Table qui enregistre les commandes de réapprovisionnement faites par les gestionnaires.

— Champs : identifiant, gestionnaire, médicament, quantité et date

— Contraintes : quantité > 0 et relations avec `Gestionnaire` et `Medicament`

— La séquence `seq_commande` gère les IDs

```
-- 6. COMMANDE
CREATE TABLE Commande (
    id_commande     NUMBER          PRIMARY KEY,
    id_gestionnaire NUMBER          NOT NULL,
    id_medicament   NUMBER          NOT NULL,
    quantite        NUMBER          NOT NULL    CHECK (quantite > 0),
    date_commande   TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_cmd_gestionnaire   FOREIGN KEY (id_gestionnaire) REFERENCES Gestionnaire(id_gestionnaire),
    CONSTRAINT fk_cmd_medicament     FOREIGN KEY (id_medicament)   REFERENCES Medicament(id_medicament)
);

CREATE SEQUENCE seq_commande START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
```

Activer Windows
Accédez aux paramètres pou

FIGURE 5.3 – table commande

5.1.4 Table StockHistorique

Table qui trace tous les mouvements de stock (entrée/sortie).

— Champs : identifiant, médicament, quantité (+ entrée, - sortie), date et type de mouvement (vente, commande, ajustement)

— La séquence `seq_historique` gère les IDs

```
-- 7. STOCK_HISTORIQUE
CREATE TABLE StockHistorique (
    id_historique    NUMBER          PRIMARY KEY,
    id_medicament    NUMBER          NOT NULL,
    quantite         NUMBER          NOT NULL,           -- + = entrée, - = sortie
    date_mouvement   TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,
    type_mouvement   VARCHAR2(20)    DEFAULT 'AUTRE',    -- VENTE, COMMANDE, AJUSTEMENT...
    CONSTRAINT fk_hist_medicament    FOREIGN KEY (id_medicament) REFERENCES Medicament(id_medicament)
);

CREATE SEQUENCE seq_historique START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
```

FIGURE 5.4 – table stockhistorique

5.2 Données de test

Chaque INSERT initialise la base avec des données de test, permettant de vérifier le bon fonctionnement des menus et des méthodes (vente, commande, consultation du stock, etc.).

```
-- Gestionnaire
INSERT INTO Gestionnaire (id_gestionnaire, nom, prenom, login, pwd)
VALUES (seq_gestionnaire.NEXTVAL, 'Martin', 'Sophie', 'gestionnaire1', 'admin');

-- Clients
INSERT INTO Client (id_client, nom, prenom, email, adresse)
VALUES (seq_client.NEXTVAL, 'Ben Ali', 'Ahmed', 'ahmed@email.com', 'Tunis');

INSERT INTO Client (id_client, nom, prenom, email, adresse)
VALUES (seq_client.NEXTVAL, 'charli', 'kirk', 'charlikirk67@email.com', 'Jandouba');
```

FIGURE 5.5 – insertion des données

Chapitre 6

Améliorations Proposées

6.1 Court Terme

6.1.1 Sécurité

— Implémenter hashage bcrypt pour mots de passe
— Ajouter timeout de session
— Logger les tentatives de connexion

6.1.2 UI/UX

— Ajouter confirmation avant opérations critiques
— Implémenter recherche par nom de médicament
— Ajouter filtres de dates dans les rapports

6.1.3 Robustesse

— Validation plus stricte des entrées utilisateur
— Messages d'erreur plus explicites
— Gestion des cas limites (divisions par zéro, etc.)

6.2 Moyen Terme

6.2.1 Fonctionnalités avancées

— Gestion des prescriptions médicales
— Intégration avec mutuelle/assurance
— Codes-barres pour scan rapide
— Export PDF des rapports

6.2.2 Architecture

— Séparation en modules (Core, UI, Data)
— API REST pour accès distant
— Application mobile complémentaire

6.2.3 Données

— Gestion des lots et dates de péremption
— Traçabilité complète (audit log)
— Backup automatique

6.3 Long Terme

6.3.1 Intelligence artificielle

— Prédiction des besoins en stock
— Détection d'anomalies de vente
— Recommandations de commande optimales

6.3.2 Intégration

— Connexion système national de santé
— Intégration comptabilité
— Interface fournisseurs automatisée

6.3.3 Cloud

— Migration vers architecture cloud
— Multi-tenancy pour plusieurs pharmacies
— Haute disponibilité et scalabilité

Chapitre 7

Conclusion Finale

Ce projet de système de gestion de pharmacie a permis de développer une application complète, fonctionnelle et évolutive. En combinant Java, JavaFX et Oracle Database, nous avons créé une solution robuste qui répond aux besoins opérationnels d'une pharmacie moderne.

L'architecture en couches, l'utilisation de patterns de conception, et la gestion rigoureuse des transactions garantissent la fiabilité du système. L'interface graphique moderne et intuitive facilite l'adoption par les utilisateurs.

Ce projet constitue une base solide qui peut être étendue et améliorée pour répondre à des besoins plus complexes. Il démontre également la capacité à concevoir, développer et documenter une application professionnelle complète.

Les défis rencontrés et les solutions apportées au cours de ce projet ont été autant d'opportunités d'apprentissage et de perfectionnement. Le résultat final illustre l'importance d'une approche méthodique, d'une conception réfléchie et d'une implémentation rigoureuse dans le développement logiciel.

« Un système bien conçu est la fondation d'une pharmacie moderne et efficace. »